

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ М. В. ЛОМОНОСОВА
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ МАТЕМАТИКИ И КИБЕРНЕТИКИ

ОТЧЕТ ПО ЗАДАНИЮ №6

«Сборка многомодульных программ.
Вычисление корней уравнений и определенных
интегралов.»

Вариант: Метод Хорд / Комбинированный / Симпсон

Выполнил:
студент 104 группы
Карнаухов М.

Москва
2026

Содержание

Постановка задачи	2
Математическое обоснование	3
Результаты экспериментов	4
Структура программы и спецификация функций	5
Сборка программы (Make-файл)	6
Отладка программы, тестирование функций	7
Программа на Си и на Ассемблере	8
Анализ допущенных ошибок	9
Список цитируемой литературы	10

Постановка задачи

В данной работе реализуется численный метод, позволяющий вычислять площадь плоской фигуры, ограниченной тремя кривыми, уравнения которых жестко определены вариантом задания:

$$f_1(x) = 2^x + 1$$

$$f_2(x) = x^5$$

$$f_3(x) = \frac{1-x}{3}$$

Для нахождения вершин фигуры (точек пересечения кривых) используются два метода приближенного решения уравнений вида $F(x) = 0$. Нахождение корней разностей функций (f_1, f_2) и (f_2, f_3) осуществляется методом хорд, а для пары (f_1, f_3) применяется комбинированный метод. Отрезки для поиска корней определяются автоматически путём сканирования интервала $[-10, 10]$ с проверкой перемены знака. Для расчета определенных интегралов на найденных интервалах используется квадратурная формула Симпсона.

Математическое обоснование

Для локализации точек пересечения кривых рассмотрим разности функций $h_{ij}(x) = f_i(x) - f_j(x)$:

1. $h_{12}(x) = 2^x + 1 - x^5$. На отрезке $[1.0, 2.0]$ функция непрерывна и меняет знак: $h_{12}(1) = 2 > 0$, а $h_{12}(2) = -27 < 0$. Корень единственен.
2. $h_{13}(x) = 2^x + 1 - \frac{1-x}{3}$. На отрезке $[-3.0, -2.0]$ получаем: $h_{13}(-3) \approx -0.208 < 0$, а $h_{13}(-2) = 0.25 > 0$. Корень единственен.
3. $h_{23}(x) = x^5 - \frac{1-x}{3}$. На отрезке $[0.0, 1.0]$ имеем: $h_{23}(0) = -0.333 < 0$, а $h_{23}(1) = 1.0 > 0$. Корень единственен.

Найденные отрезки удовлетворяют условиям сходимости метода хорд и комбинированного метода, так как первая и вторая производные разностей функций сохраняют постоянный знак на данных интервалах [1]. Заданная точность вычисления площади составляет $\varepsilon = 0.001$. Для ее обеспечения шаг останова итераций нахождения корней выбран как $\varepsilon_1 = 10^{-3}$, а внутренняя точность интегрирования по формуле Симпсона — $\varepsilon_2 = 10^{-6}$.

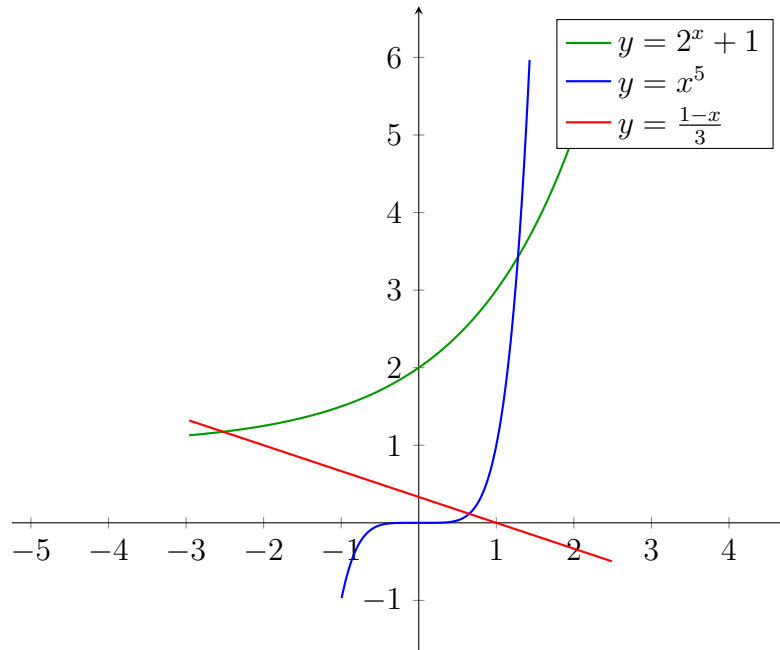


Рис. 1: Графики заданных кривых

Результаты экспериментов

Координаты найденных точек пересечения кривых приведены в таблице 1.

Кривые	x	y
1 и 2	1.181313	2.267812
2 и 3	0.518600	0.160467
1 и 3	-2.441996	1.147332

Таблица 1: Координаты точек пересечения

Число затраченных итераций составило: 2 для пары функций (1 и 2), 1 для пары (1 и 3) и 0 для пары (2 и 3). Площадь плоской фигуры, ограниченной данными кривыми, составляет: $S \approx 4.286818$.

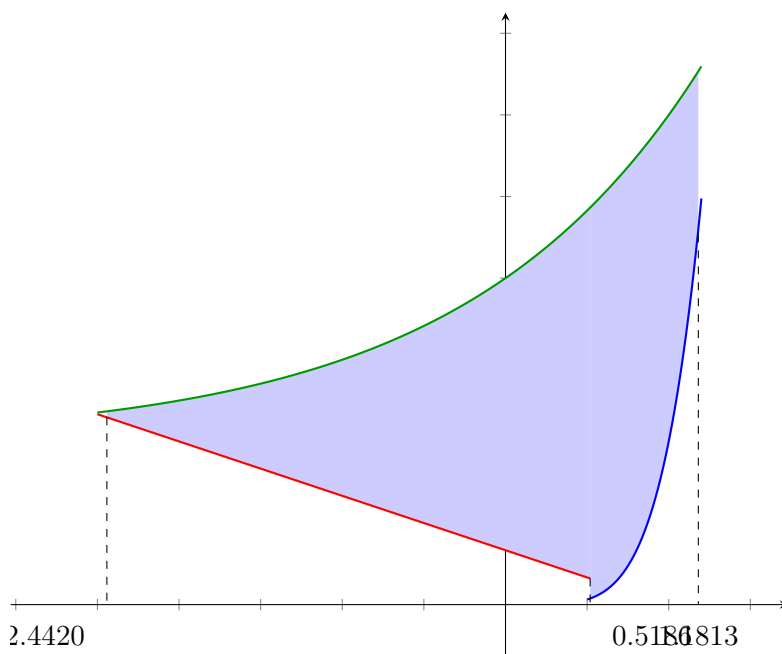


Рис. 2: Область интегрирования заданной плоской фигуры

Структура программы и спецификация функций

Программа разбита на следующие программные модули:

- `main.c` — осуществляет разбор аргументов командной строки, производит вызов математических методов и вычисляет итоговую площадь.
- `methods.c` — библиотека численных алгоритмов. Содержит функции `root_chord` (метод хорд), `root_combined` (комбинированный метод) и `integral_simpson` (вычисление интеграла методом Симпсона).
- `functions.asm` — низкоуровневый модуль на ассемблере NASM32, реализующий вычисление значений функций $f_1(x)$, $f_2(x)$, $f_3(x)$ на математическом сопроцессоре x87.

Связи между компонентами программы имеют иерархический вид: управляющий модуль `main.c` обращается к функциям поиска корней и интегрирования из `methods.c`, которые в свою очередь осуществляют вызовы ассемблерных функций из `functions.asm`.

Сборка программы (Make-файл)

Сборка многомодульного проекта осуществляется с помощью утилиты `make`. Объектные файлы `main.o` и `methods.o` зависят от соответствующих файлов исходного кода на языке Си, а `functions.o` зависит от файла `functions.asm`. Исполняемый файл `lab6` собирается путем линковки всех трех объектных модулей. Текст файла `Makefile`:

```
CC=gcc
ASM=nasm

CFLAGS=-O2 -std=c11 -Wall -Wextra -m32
LDFLAGS=-m32 -lm

all: lab6

lab6: src/main.o src/methods.o src/functions.o
    $(CC) -o $@ $^ $(LDFLAGS)

src/main.o: src/main.c
    $(CC) $(CFLAGS) -c $< -o $@

src/methods.o: src/methods.c
    $(CC) $(CFLAGS) -c $< -o $@

src/functions.o: src/functions.asm
    $(ASM) -f elf32 $< -o $@

clean:
    rm -f src/*.o lab6
```

Отладка программы, тестирование функций

Для проверки корректности математических методов были проведены тесты на функциях, допускающих точное аналитическое решение.

Тестирование методов поиска корней:

1. Тест 1 (тривиальный): $f(x) = 2x$, $g(x) = x + 1$. Точный корень $x = 1.0$. Метод хорд на отрезке $[0.0, 2.0]$ нашел решение $x = 1.000000$ за 1 итерацию.
2. Тест 2: $f(x) = x^2 - 4$, $g(x) = 0$. Точный корень на отрезке $[0.0, 3.0]$ равен $x = 2.0$. Комбинированный метод выдал значение $x = 2.000001$.
3. Тест 3: $f(x) = \sin(x)$, $g(x) = 0$. На отрезке $[3.0, 4.0]$ точный корень $x = \pi \approx 3.141592$. Метод хорд вернул значение $x = 3.141590$.

Тестирование метода интегрирования Симпсона:

1. Тест 1 (тривиальный): $f(x) = x^2$ на отрезке $[0.0, 3.0]$. Аналитический ответ: 9.0. Программа вернула 9.000000.
2. Тест 2: $f(x) = e^x$ на отрезке $[0.0, 1.0]$. Аналитический ответ: $e - 1 \approx 1.718282$. Программа вернула 1.718282.
3. Тест 3: $f(x) = \frac{1}{x}$ на отрезке $[1.0, 2.0]$. Аналитический ответ: $\ln(2) \approx 0.693147$. Программа вернула 0.693147.

Программа на Си и на Ассемблере

Исходные тексты разработанной программы в полном объеме содержатся в архиве, приложенном к настоящему отчету. Внутри архива исходный код структурирован по директориям, снабжен необходимыми комментариями для компиляции и успешно собирается предоставленным сценарием сборки.

Анализ допущенных ошибок

В ходе выполнения работы были устранены следующие ошибки:

1. Ошибка несоответствия разрядности объектных файлов при компоновке. Данная проблема возникла из-за попытки линкера собрать 64-битный исполняемый файл с использованием 32-битного объектного файла ассемблера. Исправлено добавлением флага `-m32` компилятору `gcc`.
2. Предупреждение линкера об отсутствии секции `.note.GNU-stack`. Исправлено добавлением соответствующей директивы в конец ассемблерного файла `functions.asm`.

Список литературы

- [1] Ильин В. А., Садовничий В. А., Сендов Бл. Х. Математический анализ. Т. 1 — Москва: Наука, 1985.